

Indice

Requisiti Tecnici — Integrazione VM Tunnel MPQUIC	2
1. Scopo del Documento	2
Il resto delle funzionalità richieste deve essere inteso come parte integrante del processo di integrazione della VM ROMARS all'interno dell'ecosistema TBOX.	3
2. Architettura di Riferimento	3
Flusso dati	4
3. Specifiche VM	4
Requisiti kernel	5
4. Interfacce di Rete	5
4.1 WAN — Uplink Internet (6 interfacce)	5
4.2 LAN — Collegamento verso OpenWrt (6 interfacce)	6
4.3 MGMT — Gestione (2 interfacce)	6
5. VLAN	6
5.1 VLAN su LAN4 (enp6s23)	7
5.2 VLAN su LAN5 (enp7s1)	7
5.3 VLAN su LAN6 (enp7s2)	7
5.4 VLAN su LAN1 (Interfaccia di Bonding) (enp6s20)	7
Configurazione VLAN (systemd-networkd)	7
6. Indirizzamento IP	8
6.1 Riepilogo subnet	8
6.2 Piano TUN (dettaglio)	9
7. Routing e Policy Routing	10
7.1 Tabelle di routing personalizzate	10
7.2 Routing per tabella WAN	10
7.3 Policy rules (source-based routing)	10
7.4 Routing TUN → WAN (ritorno traffico)	11
7.5 Default route	11
8. NAT e Firewall (nftables)	11
8.1 Regola NAT obbligatoria	11
8.2 IP forwarding	12
8.3 Firewall aggiuntivo	12
9. Tunnel — Nomenclatura e Addressing	12
9.1 Nomenclatura tunnel	12
9.2 Interfacce TUN	13
10. Configurazione Tunnel (YAML)	13
10.1 Schema configurazione single-path	13
10.2 Schema configurazione multi-tunnel (traffic class)	14
10.3 Schema configurazione multipath	14
10.4 Variabili ambiente (.env)	15
10.5 Parametri YAML — Lista completa	15
11. Gestione Servizi (systemd)	16
11.1 Template unit	16
11.2 Comandi di gestione	17
11.3 Script helper	17
12. Watchdog e Health-Check	18

12.1 Timer systemd	18
12.2 Logica watchdog	18
12.3 File contatore	19
13. Management REST API	19
13.1 Informazioni generali	19
13.2 Sicurezza	19
13.3 Endpoint	19
14. Metriche e Prometheus	24
14.1 Endpoint metriche per istanza	24
14.2 Porte metriche assegnate	24
14.3 Formato metriche — Single-path	24
14.4 Formato metriche — Multipath	25
14.5 Prometheus scraping	26
15. Validazione e Test	27
15.1 Test funzionali obbligatori	27
15.2 Procedura iperf3	27
15.3 Test Grafana	27
15.4 Test k6 (API load)	28
16. Supporto L3	28
16.1 Scope del supporto	28
16.2 SLA	28
16.3 Canali di comunicazione	29
17. Deliverable e Acceptance Criteria	29
17.1 Deliverable attesi	29
17.2 Acceptance Criteria	29
17.3 Timeline	29
18. VM di Test Predisposta — Note di Consegna e Accesso	30
18.1 Caratteristiche della VM	30
18.2 Configurazione di rete già presente	30
18.3 Accesso alla VM — Fase iniziale (rete locale)	32
18.4 Accesso remoto tramite ZeroTier	32
18.5 Riepilogo credenziali e riferimenti	33

Requisiti Tecnici — Integrazione VM Tunnel MPQUIC

Progetto: MPQUIC ROMARS

Data: 2026-03-26

Versione: 1.0

Classificazione: Riservato — Solo per il fornitore designato

1. Scopo del Documento

Il presente documento definisce i requisiti tecnici e di integrazione che la Virtual Machine MPQUIC fornita dal fornitore ROMARS deve soddisfare per operare correttamente

all'interno dell'architettura TBOX/OpenWrt nell'ambito del progetto pilota TRINA SOLAR.

L'obiettivo è garantire una piena interoperabilità tra la VM del fornitore e l'infrastruttura TBOX, attraverso la definizione chiara e vincolante delle interfacce esterne, includendo in particolare:

- interfacce di rete e relativa nomenclatura
- piano di indirizzamento IP
- API di gestione e controllo
- formati e modalità di esposizione delle metriche
- integrazione con i servizi di sistema (es. systemd)

Il fornitore mantiene piena libertà nella progettazione e implementazione interna della soluzione (linguaggi, librerie, architettura software), a condizione che tutte le interfacce esposte verso l'esterno risultino pienamente conformi alle specifiche definite nel presente documento.

Nel contesto del progetto pilota TRINA, l'architettura di connettività prevede esclusivamente due interfacce WAN, con i seguenti ruoli operativi:

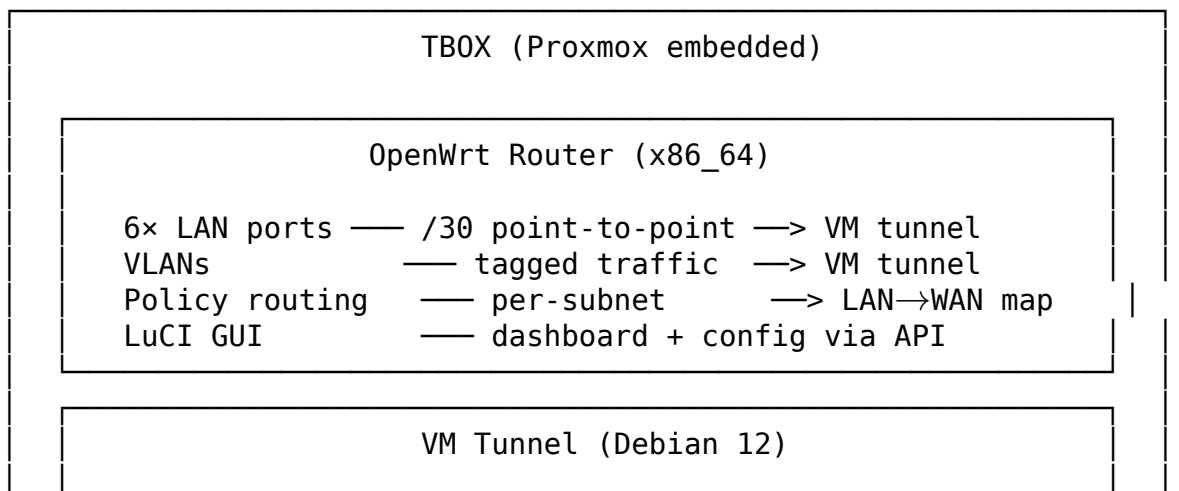
- **WAN6**: connessione primaria tramite modem Starlink (assegnazione IP via DHCP)
- **WAN5**: connessione di backup tramite modem LTE/5G (assegnazione IP via DHCP)

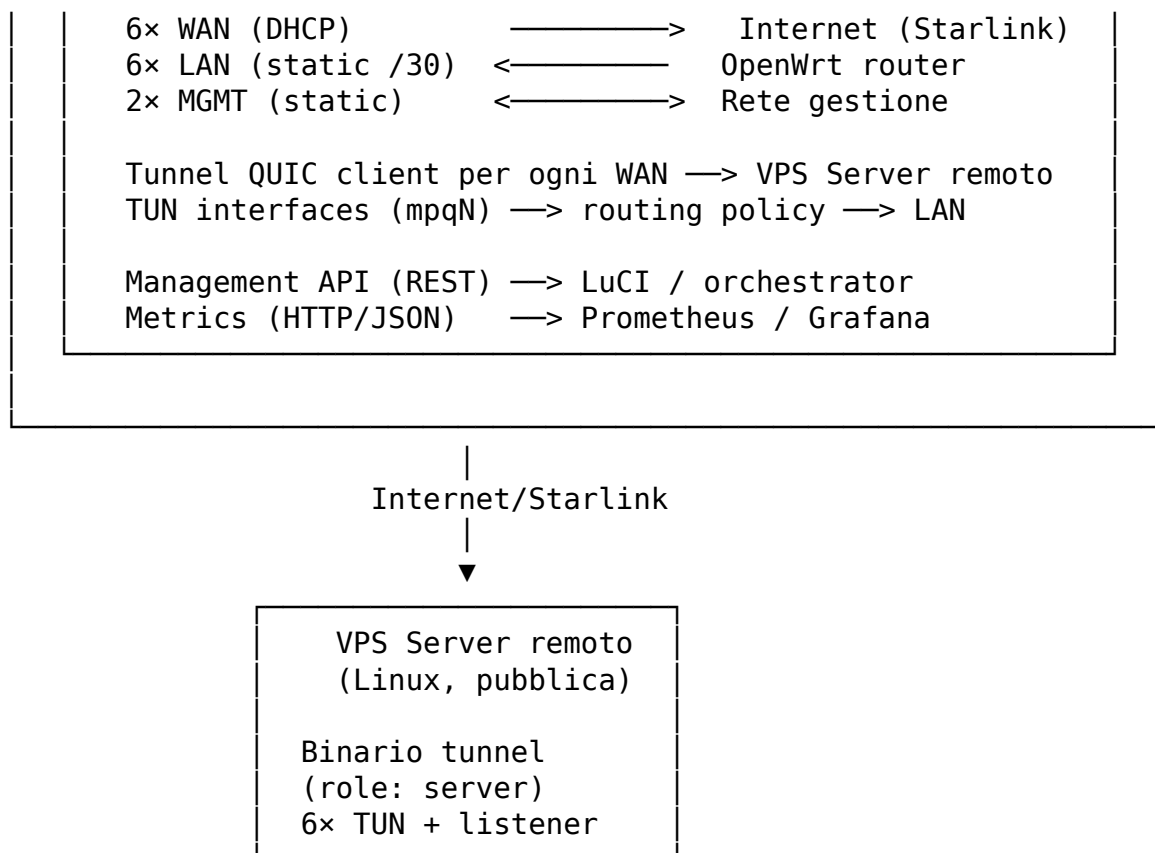
La VM MPQUIC dovrà operare in questo scenario dual-WAN garantendo continuità del servizio e corretta gestione del traffico tra i due uplink, in coerenza con le politiche definite dalla TBOX.

La soluzione dovrà basarsi esclusivamente su un'implementazione MPQUIC conforme ai requisiti di integrazione descritti nel presente documento.

Il resto delle funzionalità richieste deve essere inteso come parte integrante del processo di integrazione della VM ROMARS all'interno dell'ecosistema TBOX.

2. Architettura di Riferimento





Nel contesto del progetto pilota TRINA, l'architettura deve essere considerata in configurazione ridotta, con utilizzo esclusivo delle seguenti interfacce: wan6: uplink primario Starlink wan5: uplink secondario LTE/5G

Tutte le altre interfacce WAN, i relativi tunnel e le configurazioni multi-link sono da considerarsi fuori scope per la fase pilota e potranno essere attivate in fasi successive del progetto.

Flusso dati

1. Traffico LAN arriva alla VM tramite interfacce LAN punto-punto /30
2. Policy routing nella VM inoltra il traffico verso la TUN associata
3. Il processo tunnel incapsula i pacchetti IP in QUIC DATAGRAM utilizzando MPQUIC
4. Il socket UDP è bindata sull'interfaccia WAN corretta
5. Il pacchetto attraversa Internet verso il VPS
6. Il VPS decapsula e inoltra (o viceversa per il ritorno)

3. Specifiche VM

Parametro	Valore
Hypervisor	KVM/QEMU (libvirt)

Parametro	Valore
SO	Debian 12 (bookworm) x86_64
vCPU	Massimo 2 vCPU (assegnati: i7-10710U 1.10GHz)
RAM	Massimo 4 GB
Disco	Massimo 20 GB (SSD)
NIC fisiche (passthrough)	14 (vedi § 4)
Kernel	>= 6.1 (supporto TUN multi_queue, nftables, SO_BINDTODEVICE)
Init	systemd
Network manager	systemd-networkd (NO NetworkManager)
Firewall	nftables (NO iptables legacy)
Pacchetti minimi	iproute2, nftables, systemd, curl, jq, inetutils-ping

Requisiti kernel

- CONFIG_TUN=m (modulo tun)
- CONFIG_NF_TABLES=y (nftables)
- net.core.rmem_max >= 7340032 (7 MB — buffer socket UDP)
- net.core.wmem_max >= 7340032

4. Interfacce di Rete

La VM dispone di **14 NIC fisiche** in PCI passthrough, suddivise in 3 gruppi funzionali.

4.1 WAN — Uplink Internet (6 interfacce)

Nel progetto pilota TRINA devono essere configurate esclusivamente le seguenti interfacce:

Interfaccia	Nome logico	Connettività
enp7s7	WAN5	LTE/5G (backup)
enp7s8	WAN6	Starlink (primario)

Le restanti interfacce WAN sono da considerarsi opzionali e fuori scope per la fase pilota. Ma per completezza sono riportate qui di seguito (lo stesso vale per interfacce tun e indirizzamenti).

Interfaccia	Nome logico	Connettività
enp7s3	WAN1	Uplink ISP #1
enp7s4	WAN2	Uplink ISP #2
enp7s5	WAN3	Uplink ISP #3
enp7s6	WAN4	Uplink ISP #4

Interfaccia	Nome logico	Connettività
enp7s7	WAN5	Starlink #1
enp7s8	WAN6	Starlink #2

Nota: gli IP WAN sono dinamici (DHCP). Il software tunnel deve gestire il binding per interfaccia (SO_BINDTODEVICE) o per IP sorgente, con tolleranza al cambio IP WAN.

4.2 LAN — Collegamento verso OpenWrt (6 interfacce)

Anche per questa parte vale quanto definito sopra, la configurazione del progetto pilota TRINA prevede esclusivamente la configurazione delle seguenti interfacce: | Interfaccia | Nome logico | IP VM | IP OpenWrt | Subnet | |-----|-----|-----|-----|-----| | enp7s1 | LAN5 | 172.16.5.1 | 172.16.5.2 | /30 | | enp7s2 | LAN6 | 172.16.6.1 | 172.16.6.2 | /30 |

Le restanti interfacce LAN sono da considerarsi opzionali e fuori scope per la fase pilota.

Interfaccia	Nome logico	IP VM	IP OpenWrt	Subnet
enp6s20	LAN1	172.16.1.1	172.16.1.2	/30
enp6s21	LAN2	172.16.2.1	172.16.2.2	/30
enp6s22	LAN3	172.16.3.1	172.16.3.2	/30
enp6s23	LAN4	172.16.4.1	172.16.4.2	/30
enp7s1	LAN5	172.16.5.1	172.16.5.2	/30
enp7s2	LAN6	172.16.6.1	172.16.6.2	/30

Ogni coppia LAN forma un link point-to-point /30 tra la VM e OpenWrt.

4.3 MGMT — Gestione (2 interfacce)

Interfaccia	Rete	IP
enp6s18	Management primaria	10.10.11.100/24
enp6s19	Management secondaria	10.10.10.100/24

L'API di gestione (§ 13) deve essere raggiungibile sulla rete MGMT primaria.

5. VLAN

Il supporto alla configurazione VLAN deve essere garantito. Tuttavia, l'attivazione delle specifiche VLAN descritte nel presente documento è da considerarsi opzionale nell'ambito del progetto pilota TRINA.

Le interfacce VLAN di seguito riportate sono pertanto da ritenersi fuori scope per la fase pilota e vengono incluse esclusivamente a scopo di completezza architettuale.

Alcune interfacce LAN trasportano traffico tagged secondo lo standard IEEE 802.1Q. La VM dovrà essere in grado di gestire tali flussi creando le corrispondenti sotto-interfacce VLAN.

5.1 VLAN su LAN4 (enp6s23)

VLAN ID	Sotto-interfaccia	IP VM
11	enp6s23.11	172.16.11.1/30
12	enp6s23.12	172.16.12.1/30
13	enp6s23.13	172.16.13.1/30

5.2 VLAN su LAN5 (enp7s1)

VLAN ID	Sotto-interfaccia	IP VM
21	enp7s1.21	172.16.21.1/30
22	enp7s1.22	172.16.22.1/30
23	enp7s1.23	172.16.23.1/30

5.3 VLAN su LAN6 (enp7s2)

VLAN ID	Sotto-interfaccia	IP VM
31	enp7s2.31	172.16.31.1/30
32	enp7s2.32	172.16.32.1/30
33	enp7s2.33	172.16.33.1/30

5.4 VLAN su LAN1 (Interfaccia di Bonding) (enp6s20)

VLAN ID	Sotto-interfaccia	IP VM
17	enp6s20.17	172.16.17.1/30

Configurazione VLAN (systemd-networkd)

Per ogni VLAN serve: 1. Un file .netdev che definisce il dispositivo VLAN 2. Un file .network che assegna l'IP

Esempio per VLAN 11 su enp6s23:

```
# /etc/systemd/network/25-vlan11.netdev
```

```
[NetDev]
```

```
Name=enp6s23.11
```

```
Kind=vlan
```

```
[VLAN]
```

```
Id=11
```

```
# /etc/systemd/network/26-vlan11.network
```

```
[Match]
```

```
Name=enp6s23.11
```

```
[Network]
```

```
Address=172.16.11.1/30
```

Il parent interface (enp6s23.network) deve includere:

```
[Network]
```

```
VLAN=enp6s23.11
```

```
VLAN=enp6s23.12
```

```
VLAN=enp6s23.13
```

6. Indirizzamento IP

6.1 Riepilogo subnet

Per il progetto pilota TRINA, devono essere implementati esclusivamente i tunnel associati alle WAN attive (WAN5 e WAN6), ovvero: - **mpq5**: tunnel single-path su WAN5 (LTE/5G, backup) - **mpq6**: tunnel single-path su WAN6 (Starlink, primario) - **mp1**: tunnel multipath/bonding su WAN5+WAN6 con policy **failover** (WAN6 primaria, WAN5 backup)

Configurazione Progetto Pilota:

Ruolo	Subnet	Gateway	Note
WAN1-6	DHCP	DHCP-assegnato	IP dinamici
LAN1	172.16.1.0/30	—	point-to-point
LAN2	172.16.2.0/30	—	point-to-point
LAN3	172.16.3.0/30	—	point-to-point
LAN4	172.16.4.0/30	—	point-to-point
LAN5	172.16.5.0/30	—	point-to-point
LAN6	172.16.6.0/30	—	point-to-point
MGMT1	10.10.11.0/24	10.10.11.254	gestione
MGMT2	10.10.10.0/24	—	gestione secondaria
TUN 1-6	10.200.{1-6}.0/30	—	tunnel single-path

Configurazione mancante Template:

Ruolo	Subnet	Gateway	Note
VLAN 11	172.16.11.0/30	—	su LAN4
VLAN 12	172.16.12.0/30	—	su LAN4
VLAN 13	172.16.13.0/30	—	su LAN4
VLAN 21	172.16.21.0/30	—	su LAN5
VLAN 22	172.16.22.0/30	—	su LAN5
VLAN 23	172.16.23.0/30	—	su LAN5
VLAN 31	172.16.31.0/30	—	su LAN6
VLAN 32	172.16.32.0/30	—	su LAN6
VLAN 33	172.16.33.0/30	—	su LAN6
VLAN 17	172.16.17.0/30	—	su LAN1
TUN cr/br/df	10.200.{14-16}.0/24	—	tunnel multi-class
TUN mp1	10.200.17.0/24	—	tunnel multipath

6.2 Piano TUN (dettaglio)

Configurazione Progetto Pilota:

Tunnel	TUN name	Client IP	Server IP	Porta server
1	mpq1	10.200.1.1	10.200.1.2	45001
2	mpq2	10.200.2.1	10.200.2.2	45002
3	mpq3	10.200.3.1	10.200.3.2	45003
4	mpq4	10.200.4.1	10.200.4.2	45004
5	mpq5	10.200.5.1	10.200.5.2	45005
6	mpq6	10.200.6.1	10.200.6.2	45006

Configurazione mancante Template:

Tunnel	TUN name	Client IP	Server IP	Porta server
cr4	tun-cr4	10.200.14.1	—	45014
br4	tun-br4	10.200.14.5	—	45014
df4	tun-df4	10.200.14.9	—	45014
cr5	tun-cr5	10.200.15.1	—	45015
br5	tun-br5	10.200.15.5	—	45015
df5	tun-df5	10.200.15.9	—	45015
cr6	tun-cr6	10.200.16.1	—	45016
br6	tun-br6	10.200.16.5	—	45016
df6	tun-df6	10.200.16.9	—	45016
mp1	mp1	10.200.17.1	10.200.17.254	46017

Nota: la configurazione dei tunnel cr/br/df (traffic-class) per lo stesso link è a carico del fornitore

7. Routing e Policy Routing

7.1 Tabelle di routing personalizzate

La VM deve definire le seguenti tabelle in /etc/iproute2/rt_tables:

```
# Single-path tunnel routing
100    wan1
101    wan2
102    wan3
103    wan4
104    wan5
105    wan6

# Multi-tunnel class routing (opzionale fase 2)
120    mt_cr4
121    mt_br4
122    mt_df4
123    mt_cr5
124    mt_br5
125    mt_df5
126    mt_cr6
127    mt_br6
128    mt_df6

# Multipath bonding
200    bd1
```

7.2 Routing per tabella WAN

Per ogni WAN N (1-6), il traffico verso il VPS deve uscire dalla WAN corretta.

```
# Esempio per WAN5 (enp7s7)
ip route add default via <GW_WAN5> dev enp7s7 table wan5
```

Il gateway WAN è assegnato via DHCP e deve essere scoperto dinamicamente.

7.3 Policy rules (source-based routing)

Ogni subnet LAN interna che deve usare un tunnel specifico ha una regola:

```
# Traffico dalla LAN5 -> esce su WAN5
ip rule add from 172.16.5.0/30 lookup wan5 priority 100

# Traffico dalla LAN6 -> esce su WAN6
ip rule add from 172.16.6.0/30 lookup wan6 priority 100
```

7.4 Routing TUN → WAN (ritorno traffico)

Per ogni tunnel single-path, il traffico di ritorno dal VPS entra sulla TUN e deve raggiungere la LAN corretta:

```
# Il server VPS (10.200.5.2) raggiunga il client via WAN5
ip route add 10.200.5.2 via <GW_WAN5> dev enp7s7 table wan5
ip rule add from 10.200.5.0/30 lookup wan5 priority 50
```

7.5 Default route

La tabella main deve avere almeno una default route verso una WAN attiva:

```
ip route add default via <GW_WANX> dev enp7sX
```

La scelta della WAN default è a discrezione dell'implementazione.

8. NAT e Firewall (nftables)

8.1 Regola NAT obbligatoria

Tutto il traffico in uscita sulle interfacce WAN e tunnel deve essere mascherato (SNAT).

```
table ip nat {
    chain postrouting {
        type nat hook postrouting priority srcnat; policy accept;

        # WAN interfaces
        oifname "enp7s3" masquerade
        oifname "enp7s4" masquerade
        oifname "enp7s5" masquerade
        oifname "enp7s6" masquerade
        oifname "enp7s7" masquerade
        oifname "enp7s8" masquerade

        # Tunnel TUN interfaces (single-path)
        oifname "mpq1" masquerade
        oifname "mpq2" masquerade
        oifname "mpq3" masquerade
        oifname "mpq4" masquerade
        oifname "mpq5" masquerade
        oifname "mpq6" masquerade

        # Tunnel TUN interfaces (multi-class)
        oifname "tun-cr4" masquerade
        oifname "tun-br4" masquerade
        oifname "tun-df4" masquerade
        oifname "tun-cr5" masquerade
    }
}
```

```

        oifname "tun-br5" masquerade
        oifname "tun-df5" masquerade
        oifname "tun-cr6" masquerade
        oifname "tun-br6" masquerade
        oifname "tun-df6" masquerade

        # Multipath
        oifname "mp1" masquerade
    }
}

```

8.2 IP forwarding

Deve essere abilitato:

```
sysctl -w net.ipv4.ip_forward=1
```

Persistente in `/etc/sysctl.d/99-mpquic.conf`:

```

net.ipv4.ip_forward = 1
net.core.rmem_max = 7340032
net.core.wmem_max = 7340032

```

8.3 Firewall aggiuntivo

Nessuna regola filter aggiuntiva è richiesta in questa fase. Il traffico tra LAN, TUN e WAN deve poter transitare liberamente.

9. Tunnel — Nomenclatura e Addressing

La nomenclatura e l'indirizzamento descritti in questo paragrafo rappresentano le best practice dell'architettura target. Per il progetto pilota TRINA, il fornitore è tenuto a implementare esclusivamente i tunnel mpq5, mpq6 e mp1; per il naming e l'indirizzamento degli stessi è fortemente consigliato seguire le indicazioni riportate di seguito, al fine di garantire piena compatibilità con i sistemi di monitoraggio e gestione già predisposti (Prometheus, Grafana, LuCI).

9.1 Nomenclatura tunnel

Il sistema di naming è **fisso** e deve essere rispettato esattamente.

Single-path (1 tunnel per WAN)

Istanza	Nome TUN	WAN	Porta VPS	TUN CIDR
1	mpq1	enp7s3	45001	10.200.1.1/30

Istanza	Nome TUN	WAN	Porta VPS	TUN CIDR
2	mpq2	enp7s4	45002	10.200.2.1/30
3	mpq3	enp7s5	45003	10.200.3.1/30
4	mpq4	enp7s6	45004	10.200.4.1/30
5	mpq5	enp7s7	45005	10.200.5.1/30
6	mpq6	enp7s8	45006	10.200.6.1/30

Multi-tunnel per link (3 classi traffico per WAN)

Istanza	Nome TUN	WAN	Porta VPS	TUN CIDR	Classe
cr4	tun-cr4	enp7s6	45014	10.200.14.1/30	critical
br4	tun-br4	enp7s6	45014	10.200.14.5/30	browsing
df4	tun-df4	enp7s6	45014	10.200.14.9/30	default
cr5	tun-cr5	enp7s7	45015	10.200.15.1/30	critical
br5	tun-br5	enp7s7	45015	10.200.15.5/30	browsing
df5	tun-df5	enp7s7	45015	10.200.15.9/30	default
cr6	tun-cr6	enp7s8	45016	10.200.16.1/30	critical
br6	tun-br6	enp7s8	45016	10.200.16.5/30	browsing
df6	tun-df6	enp7s8	45016	10.200.16.9/30	default

Multipath bonding

Istanza	Nome TUN	WAN	Porta VPS	TUN CIDR
mp1	mp1	enp7s6+7+8	46017	10.200.17.1/24

9.2 Interfacce TUN

Ogni tunnel richiede un dispositivo TUN Linux dedicato. Il dispositivo deve:

- Essere creato prima dell'avvio del processo tunnel
- Avere MTU configurabile (default: 1280)
- Supportare `multi_queue` per prestazioni multi-core
- Avere l'IP assegnato come da piano (§ 6.2)

10. Configurazione Tunnel (YAML)

Ogni istanza tunnel deve avere un file YAML di configurazione in `/etc/mpquic/instances/`. Il nome del file identifica l'istanza.

10.1 Schema configurazione single-path

```
# /etc/mpquic/instances/5.yaml
role: client
bind_ip: "if:enp7s7"           # Binding per interfaccia (SO_BINDTODEVICE)
remote_addr: "<VPS_PUBLIC_IP>" # IP pubblico del server VPS
remote_port: 45005
tun_name: mpq5
tun_cidr: "10.200.5.1/30"
tun_mtu: 1280
log_level: info
metrics_port: 9095             # HTTP metrics (porta unica per istanza)
```

10.2 Schema configurazione multi-tunnel (traffic class)

```
# /etc/mpquic/instances/cr5.yaml
role: client
bind_ip: "if:enp7s7"
remote_addr: "<VPS_PUBLIC_IP>"
remote_port: 45015
tun_name: tun-cr5
tun_cidr: "10.200.15.1/30"
tun_mtu: 1280
log_level: info
metrics_port: 9115
```

10.3 Schema configurazione multipath

```
# /etc/mpquic/instances/mp1.yaml
role: client
tun_name: mp1
tun_cidr: "10.200.17.1/24"
tun_mtu: 1280
multipath_enabled: true
multipath_policy: failover # priority | failover | balanced
log_level: info
metrics_port: 9117

multipath_paths:
- name: wan6
  bind_ip: "if:enp7s8"
  remote_addr: "<VPS_PUBLIC_IP>"
  remote_port: 46017
  priority: 10 # priorità più alta (valore numerico più basso)
  weight: 1
  transport: quic
```

```
- name: wan5
  bind_ip: "if:enp7s7"
  remote_addr: "<VPS_PUBLIC_IP>"
  remote_port: 46017
  priority: 100
  weight: 1
  transport: quic
```

backup – usato solo se wan6 non disponibile

10.4 Variabili ambiente (.env)

Ogni istanza può avere un file .env associato in /etc/mpquic/instances/:

```
# /etc/mpquic/instances/5.env
TUN_NAME=mpq5
TUN_CIDR=10.200.5.1/30
TUN_PEER=10.200.5.2
TUN_MTU=1280
VPS_PUBLIC_IP=<indirizzo_vps>
```

Un file global.env definisce variabili comuni a tutte le istanze:

```
# /etc/mpquic/instances/global.env
VPS_PUBLIC_IP=<indirizzo_vps>
```

10.5 Parametri YAML — Lista completa

Parametro	Tipo	Obbligatorio	Descrizione
role	string	Sì	client o server
bind_ip	string	Sì (client)	IP o if:<ifname> per SO_BINDTODEVICE
remote_addr	string	Sì (client)	IP del server VPS
remote_port	int	Sì	Porta UDP server (es. 45005)
listen_addr	string	Sì (server)	Indirizzo di ascolto (es. 0.0.0.0)
listen_port	int	Sì (server)	Porta di ascolto
tun_name	string	Sì	Nome interfaccia TUN
tun_cidr	string	Sì	CIDR dell'interfaccia TUN
tun_mtu	int	No	MTU della TUN (default: 1280)
log_level	string	No	debug, info, warn, error

Parametro	Tipo	Obbligatorio	Descrizione
metrics_port	int	No	Porta HTTP metriche (0 = disabilitato)
multipath_enabled	bool	No	Abilita modalità multipath
multipath_policy	string	No	priority, failover, balanced
multipath_paths	array	Se multipath	Lista path (vedi § 10.3)
congestion	string	No	bbr, cubic (default: bbr)

11. Gestione Servizi (systemd)

11.1 Template unit

Il software tunnel deve essere gestito tramite un systemd template unit mpquic@.service che permette istanze multiple:

```
[Unit]
Description=MPQUIC tunnel instance %i
After=network-online.target
Wants=network-online.target
StartLimitIntervalSec=300
StartLimitBurst=5

[Service]
Type=simple
EnvironmentFile=-/etc/mpquic/instances/global.env
EnvironmentFile=-/etc/mpquic/instances/%i.env

# Pre-start: crea/configura TUN e renderizza config
ExecStartPre=/usr/local/bin/ensure_tun.sh
ExecStartPre=/usr/local/bin/render_config.sh /etc/mpquic/instances/%i.yaml.tpl /etc/

ExecStart=/usr/local/bin/mpquic -config /etc/mpquic/instances/%i.yaml

Restart=always
RestartSec=3
LimitNOFILE=65535
AmbientCapabilities=CAP_NET_ADMIN CAP_NET_RAW CAP_NET_BIND_SERVICE
```



```
[Install]
WantedBy=multi-user.target
```

11.2 Comandi di gestione

```
# Avvio singola istanza
systemctl start mpquic@5

# Stop singola istanza
systemctl stop mpquic@5

# Restart
systemctl restart mpquic@5

# Stato
systemctl is-active mpquic@5

# Abilitazione al boot
systemctl enable mpquic@5

# Log
journalctl -u mpquic@5 -f
```

11.3 Script helper

ensure_tun.sh Script idempotente che crea la TUN se non esiste, assegna IP e MTU. Legge le variabili TUN_NAME, TUN_CIDR, TUN_MTU dall'ambiente.

```
#!/bin/bash
set -euo pipefail
: "${TUN_NAME:?}"
: "${TUN_CIDR:?}"
: "${TUN_MTU:=1280}"

if ! ip link show "$TUN_NAME" &>/dev/null; then
    ip tuntap add dev "$TUN_NAME" mode tun multi_queue
fi

ip addr flush dev "$TUN_NAME" 2>/dev/null || true
ip addr add "$TUN_CIDR" dev "$TUN_NAME"
ip link set "$TUN_NAME" mtu "$TUN_MTU" up
```

render_config.sh Script che sostituisce variabili nel template YAML:

```
#!/bin/bash
set -euo pipefail
```

```
TPL="$1"
OUT="$2"
[[ -f "$TPL" ]] || exit 0
envsubst < "$TPL" > "$OUT"
```

12. Watchdog e Health-Check

12.1 Timer systemd

Un timer systemd deve eseguire un check periodico (ogni 60 secondi):

```
# /etc/systemd/system/mpquic-watchdog.timer
```

```
[Unit]
```

```
Description=MPQUIC tunnel watchdog timer
```

```
[Timer]
```

```
OnBootSec=120
```

```
OnUnitActiveSec=60
```

```
RandomizedDelaySec=5
```

```
[Install]
```

```
WantedBy=timers.target
```

```
# /etc/systemd/system/mpquic-watchdog.service
```

```
[Unit]
```

```
Description=MPQUIC tunnel watchdog
```

```
[Service]
```

```
Type=oneshot
```

```
ExecStart=/usr/local/bin/mpquic-tunnel-watchdog.sh
```

12.2 Logica watchdog

Lo script di watchdog deve implementare i seguenti controlli per ogni istanza tunnel abilitata:

1. **WAN carrier check:** verifica che l'interfaccia WAN associata abbia carrier (cat /sys/class/net/<ifname>/carrier)
2. **TUN existence check:** verifica che l'interfaccia TUN esista e sia UP
3. **Peer reachability:** ping verso il peer tunnel (es. 10.200.5.2) con timeout breve
4. **Threshold-based restart:** restart dell'istanza solo dopo N fallimenti consecutivi (consigliato: 3), per evitare flapping

Per ogni istanza abilitata:

1. Se WAN non ha carrier → skip (non riavviare, attenderne il ripristino)
2. Se TUN non esiste → restart immediato
3. Se peer non risponde → incrementa contatore

- Se contatore $\geq$ THRESHOLD $\rightarrow$ restart + reset contatore
- Altrimenti $\rightarrow$ log warning e attesa
- 4. Se peer risponde $\rightarrow$ reset contatore

12.3 File contatore

Il contatore di fallimenti deve essere persistito per sopravvivere a restart del watchdog.
Formato consigliato:

/run/mpquic-watchdog/<instance>.fail_count

13. Management REST API

La VM deve esporre una REST API HTTP(S) sulla rete di gestione, compatibile con il contratto definito di seguito.

13.1 Informazioni generali

Parametro	Valore
Bind address	10.10.11.100:8080 (MGMT primaria)
Protocollo	HTTP (fase 1), HTTPS/TLS 1.2+ (fase 2)
Autenticazione	Bearer token (Authorization: Bearer <token>)
Content-Type	application/json
CORS	Opzionale, configurabile

13.2 Sicurezza

- Il token deve avere lunghezza minima 16 caratteri
- Il confronto del token deve essere constant-time (prevenzione timing attack)
- Rate limiting: max 10 tentativi falliti per IP in finestra di 5 minuti
- Header di sicurezza obbligatori nelle risposte:
 - X-Content-Type-Options: nosniff
 - X-Frame-Options: DENY
 - Cache-Control: no-store
- Input validation: i nomi tunnel devono matchare $^{[a-zA-Z0-9]}[a-zA-Z0-9._-]{0,63}\$$
- Body size limit: 512 KB

13.3 Endpoint

GET /api/v1/health Health check globale.

Risposta (200 OK):

```
{
  "ok": true,
  "version": "1.0.0",
  "hostname": "tbox-mpquic",
  "os": "linux",
  "arch": "amd64",
  "tunnels_total": 16,
  "tunnels_running": 4,
  "tunnels_stopped": 12,
  "tunnels_failed": 0,
  "timestamp": "2026-06-11T10:00:00Z"
}
```

GET /api/v1/tunnels Lista tutte le istanze tunnel con stato corrente.

Risposta (200 OK):

```
{
  "tunnels": [
    {
      "name": "5",
      "status": "running",
      "uptime_sec": 3600,
      "tun_name": "mpq5",
      "tun_cidr": "10.200.5.1/30",
      "transport": "quic",
      "config_file": "/etc/mpquic/instances/5.yaml"
    },
    {
      "name": "mp1",
      "status": "running",
      "uptime_sec": 7200,
      "tun_name": "mp1",
      "tun_cidr": "10.200.17.1/24",
      "transport": "quic",
      "config_file": "/etc/mpquic/instances/mp1.yaml"
    }
  ]
}
```

Campi obbligatori per ogni tunnel:

Campo	Tipo	Descrizione
name	string	Nome istanza (es. 5, cr5, mp1)
status	string	running, stopped, failed
uptime_sec	int	Secondi dall'avvio (0 se non running)
tun_name	string	Nome interfaccia TUN

Campo	Tipo	Descrizione
tun_cidr	string	CIDR assegnato alla TUN
transport	string	quic
config_file	string	Path assoluto al file YAML

GET /api/v1/tunnels/{name} Dettaglio singola istanza. Stessi campi di cui sopra.

POST /api/v1/tunnels/{name}/start Avvia l'istanza tunnel.

Risposta (200 OK):

```
{
  "ok": true,
  "action": "start",
  "tunnel": "5",
  "status": "running"
}
```

POST /api/v1/tunnels/{name}/stop Ferma l'istanza tunnel. Stesso formato risposta.

POST /api/v1/tunnels/{name}/restart Riavvia l'istanza tunnel. Stesso formato risposta.

GET /api/v1/tunnels/{name}/config Legge la configurazione corrente dell'istanza.

Risposta (200 OK):

```
{
  "tunnel": "5",
  "config": {
    "role": "client",
    "bind_ip": "if:enp7s7",
    "remote_addr": "1.2.3.4",
    "remote_port": 45005,
    "tun_name": "mpq5",
    "tun_cidr": "10.200.5.1/30",
    "tun_mtu": 1280,
    "log_level": "info",
    "metrics_port": 9095
  },
  "param_categories": {
    "A": ["log_level", "metrics_port"],
    "B": ["tun_mtu", "congestion"],
    "C": ["role", "bind_ip", "remote_addr", "remote_port", "tun_name", "tun_cidr"]
  }
}
```

```
}  
}
```

Categorie parametro: - **A (hot-reload)**: modificabili senza restart - **B (restart)**: richiedono restart dell'istanza - **C (server-coupled)**: richiedono coordinamento con il server VPS, modifica bloccata via API

PATCH /api/v1/tunnels/{name}/config Modifica parziale della configurazione.

Per client che non supportano PATCH (es. wget, uclient-fetch), è supportato: POST con header X-HTTP-Method-Override: PATCH.

Request body:

```
{  
  "log_level": "debug",  
  "tun_mtu": 1400  
}
```

Risposta (200 OK):

```
{  
  "ok": true,  
  "tunnel": "5",  
  "fields_applied": ["log_level", "tun_mtu"],  
  "needs_restart": true,  
  "restart_applied": false  
}
```

Query parameter opzionale: ?auto_restart=true per trigger automatico del restart.

POST /api/v1/tunnels/{name}/config/validate Valida un patch di configurazione senza applicarlo.

Risposta (200 OK se valido):

```
{  
  "ok": true,  
  "needs_restart": true  
}
```

Risposta (400 Bad Request se invalido):

```
{  
  "ok": false,  
  "error": "field 'remote_addr' is category C (server-coupled), modification blocked",  
  "blocked_fields": ["remote_addr"],  
  "needs_restart": false  
}
```

GET /api/v1/tunnels/{name}/metrics Proxy verso le metriche dell'istanza tunnel.

Risposta: passthrough del JSON restituito dall'endpoint metriche dell'istanza (vedi § 14).

GET /api/v1/tunnels/{name}/logs?lines=100&level=error Ultimi log dell'istanza da journald.

Query param	Default	Descrizione
lines	100	Numero righe (max 10000)
level	(tutti)	Filtra: error, warning

Risposta (200 OK):

```
{
  "tunnel": "5",
  "lines": 100,
  "level": "error",
  "output": "Jun 11 10:00:01 tbox mpquic[1234]: ERROR: ..."
}
```

GET /api/v1/metrics Metriche aggregate di tutti i tunnel.

Risposta (200 OK):

```
{
  "tunnels": {
    "5": { "total_tx_bytes": 123456, "total_rx_bytes": 789012, ... },
    "6": { "total_tx_bytes": 1000, "total_rx_bytes": 2000, ... },
    "mp1": { "total_tx_bytes": 5000000, "total_rx_bytes": 8000000, ... }
  }
}
```

GET /api/v1/system/info Informazioni di sistema.

Risposta (200 OK):

```
{
  "mgmt_version": "1.0.0",
  "mpquic_version": "v2.3.0",
  "git_commit": "abc1234 fix: ...",
  "version": "1.22.0",
  "hostname": "tbox-mpquic",
  "os": "linux",
  "arch": "amd64",
  "num_cpu": 2,
  "uptime": "up 5 days, 3 hours",
}
```

```
"timestamp": "2026-06-11T10:00:00Z"
}
```

GET `/api/v1/system/logs/{name}?lines=100&level=error` Stessa semantica di `/api/v1/tunnels/{name}/logs` (endpoint alternativo).

14. Metriche e Prometheus

14.1 Endpoint metriche per istanza

Ogni istanza tunnel deve esporre un endpoint HTTP JSON sulla porta `metrics_port` configurata nel YAML.

Endpoint: GET `http://127.0.0.1:<metrics_port>/api/v1/stats`

14.2 Porte metriche assegnate

Istanza	Porta
1	9091
2	9092
3	9093
4	9094
5	9095
6	9096
cr4	9114
br4	9114
df4	9114
cr5	9115
br5	9115
df5	9115
cr6	9116
br6	9116
df6	9116
mp1	9117

Nota: i tunnel multi-class che condividono lo stesso server process possono condividere la stessa porta metriche.

14.3 Formato metriche — Single-path

```
{
  "tunnel": "5",
  "role": "client",
  "status": "running",
}
```



```

"uptime_sec": 3600,
"tun_name": "mpq5",
"tun_cidr": "10.200.5.1/30",
"transport": "quic",
"global": {
  "total_tx_bytes": 123456,
  "total_rx_bytes": 789012,
  "total_tx_pkts": 1023,
  "total_rx_pkts": 6542,
  "tx_rate_bps": 1234567,
  "rx_rate_bps": 7890123,
  "uptime_seconds": 3600
}
}

```

14.4 Formato metriche — Multipath

```

{
  "tunnel": "mpl",
  "role": "client",
  "status": "running",
  "uptime_sec": 7200,
  "tun_name": "mpl",
  "tun_cidr": "10.200.17.1/24",
  "transport": "quic",
  "paths": [
    {
      "name": "wan6",
      "state": "up",
      "tx_bytes": 1500000,
      "rx_bytes": 2500000,
      "tx_pkts": 800,
      "rx_pkts": 1200,
      "rtt_us": 25000,
      "loss_pct": 0.1,
      "consecutive_fails": 0
    },
    {
      "name": "wan5",
      "state": "standby",
      "tx_bytes": 0,
      "rx_bytes": 0,
      "tx_pkts": 0,
      "rx_pkts": 0,
      "rtt_us": 45000,
      "loss_pct": 0.0,
    }
  ]
}

```

```

        "consecutive_fails": 0
    }
],
"global": {
    "total_tx_bytes": 1500000,
    "total_rx_bytes": 2500000,
    "total_tx_pkts": 800,
    "total_rx_pkts": 1200,
    "active_paths": 1,
    "total_paths": 2,
    "uptime_seconds": 7200
}
}

```

14.5 Prometheus scraping

Le metriche devono essere compatibili con scraping Prometheus. L'approccio consigliato è un exporter esterno che converte il JSON in formato Prometheus text exposition, oppure un endpoint nativo /metrics in formato Prometheus.

Metriche minime attese (naming convention Prometheus):

```

# HELP mpquic_tunnel_tx_bytes_total Total bytes transmitted
# TYPE mpquic_tunnel_tx_bytes_total counter
mpquic_tunnel_tx_bytes_total{instance="5",tun="mpq5"} 123456

# HELP mpquic_tunnel_rx_bytes_total Total bytes received
# TYPE mpquic_tunnel_rx_bytes_total counter
mpquic_tunnel_rx_bytes_total{instance="5",tun="mpq5"} 789012

# HELP mpquic_tunnel_tx_pkts_total Total packets transmitted
# TYPE mpquic_tunnel_tx_pkts_total counter
mpquic_tunnel_tx_pkts_total{instance="5",tun="mpq5"} 1023

# HELP mpquic_tunnel_rx_pkts_total Total packets received
# TYPE mpquic_tunnel_rx_pkts_total counter
mpquic_tunnel_rx_pkts_total{instance="5",tun="mpq5"} 6542

# HELP mpquic_tunnel_up Tunnel operational status (1=up, 0=down)
# TYPE mpquic_tunnel_up gauge
mpquic_tunnel_up{instance="5",tun="mpq5"} 1

# HELP mpquic_path_state Multipath path state (1=up, 0=down)
# TYPE mpquic_path_state gauge
mpquic_path_state{instance="mp1",path="wan6"} 1
mpquic_path_state{instance="mp1",path="wan5"} 0

```

15. Validazione e Test

15.1 Test funzionali obbligatori

I test che coinvolgono tunnel che non siano quelli necessari alla fase pilota sono opzionali

#	Test	Criterio di successo
T1	Connettività single-path	Ping dal VPS verso 10.200.N.1 per N=1..6 con RTT < 200ms
T2	Throughput single-path	iperf3 TCP attraverso tunnel: >= 80 Mbps per link Starlink
T3	Isolamento traffico	Loss artificiale (tc netem 10%) su un tunnel non impatta i tunnel adiacenti
T4	Failover multipath	Disconnect WAN6 (Starlink): traffico migra su WAN5 (LTE) in < 15s, packet loss < 5%
T5	Failover recovery	Ripristino WAN6: traffico torna su WAN6 (primaria) entro 30s, automaticamente
T6	Watchdog recovery	Kill processo mpquic@5: watchdog lo riavvia entro 120s
T7	API health	GET /api/v1/health ritorna 200 con ok: true
T8	API tunnel lifecycle	stop/start/restart via API cambiano stato systemd
T9	Metriche non-zero	Dopo traffico iperf3, TX/RX bytes > 0 per ogni tunnel attivo
T10	NAT masquerade	Traffico dalla LAN OpenWrt esce verso Internet con IP WAN

15.2 Procedura iperf3

```
# Sul VPS (server iperf3)
iperf3 -s -p 5201

# Sulla VM tunnel (client iperf3) – test tunnel 5
iperf3 -c 10.200.5.2 -p 5201 -t 30 -P 4

# Test multipath mp1
iperf3 -c 10.200.17.254 -p 5201 -t 30 -P 8
```

15.3 Test Grafana

Questo paragrafo è opzionale

Il fornitore deve dimostrare che le metriche sono visibili su una dashboard Grafana con: - Pannello per-tunnel con TX/RX bytes rate - Pannello stato tunnel (running/stopped/failed) - Pannello path multipath (stato, RTT, loss)

15.4 Test k6 (API load)

L'API di gestione deve sostenere almeno: - 100 richieste/secondo su GET /api/v1/health
- 50 richieste/secondo su GET /api/v1/tunnels - Risposta P95 < 200ms

Esempio script k6:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  vus: 10,
  duration: '60s',
};

const BASE = 'http://10.10.11.100:8080';
const TOKEN = '<auth_token>';

export default function () {
  const res = http.get(`${BASE}/api/v1/health`, {
    headers: { Authorization: `Bearer ${TOKEN}` },
  });
  check(res, {
    'status 200': (r) => r.status === 200,
    'body ok': (r) => JSON.parse(r.body).ok === true,
  });
}
```

16. Supporto L3

16.1 Scope del supporto

Il fornitore deve garantire supporto di Livello 3 (L3) per i seguenti componenti:

Componente	Descrizione
Software tunnel	Bug fix, patch di sicurezza, aggiornamenti funzionali
Management API	Manutenzione endpoint, fix compatibilità
Watchdog	Correzioni logica health-check
Configurazione rete	Assistenza su routing, NAT, VLAN

16.2 SLA

Gli SLA faranno parte di un contratto a parte e perciò non sono applicabili.

Metrica	Valore target
Tempo di risposta (P1 — tunnel down)	< 4 ore lavorative
Tempo di risposta (P2 — degradazione)	< 8 ore lavorative
Tempo di risposta (P3 — richiesta info)	< 2 giorni lavorativi
Uptime tunnel (target mensile)	>= 99.5%
Manutenzione preventiva	Inclusa
Aggiornamento software	Trimestrale (minimo)

16.3 Canali di comunicazione

- Ticketing via piattaforma condivisa (JIRA/GitLab/equivalente)
- Accesso SSH alla VM per troubleshooting remoto
- Documentazione aggiornamenti via changelog versionato

17. Deliverable e Acceptance Criteria

17.1 Deliverable attesi

#	Deliverable	Formato
D1	Immagine VM Debian 12 pre-configurata	QCOW2 / OVA
D2	Binario tunnel (client + server)	Eseguibile Linux amd64
D3	Configurazione systemd	File .service + .timer
D4	Script helper (ensure_tun, watchdog, render)	Bash scripts
D5	Management API daemon	Eseguibile Linux amd64
D6	Documentazione tecnica	Markdown
D7	Risultati test (§ 15)	Report con evidenze

17.2 Acceptance Criteria

La VM fornita sarà accettata quando:

1. **Tutti i 10 test funzionali (T1-T10)** sono superati
2. **L'API risponde correttamente** a tutti gli endpoint definiti in § 13
3. **Le metriche** sono coerenti con il formato definito in § 14
4. **La nomenclatura** di interfacce, tunnel, file config rispetta esattamente quanto definito in §§ 4, 9, 10
5. **Il watchdog** rileva e recupera automaticamente un tunnel in errore
6. **La VM si avvia al boot** con tutti i tunnel configurati attivi
7. **Il piano di indirizzamento** (§ 6) è rispettato senza deviazioni
8. **Le regole NAT** (§ 8) sono funzionanti e verificate con traceroute

17.3 Timeline

Fase	Descrizione	Durata stimata
Fase 1	Single-path (6 tunnel) + API + metriche	4 settimane
Fase 2	Multi-tunnel traffic-class (9 tunnel)	1 settimane
Fase 3	Multipath bonding	1 settimane
Fase 4	Integration test + acceptance	2 settimane

18. VM di Test Predisposta — Note di Consegna e Accesso

Telespazio ha predisposto una macchina virtuale (VM) pronta all'uso da impiegare come ambiente di sviluppo e test per l'implementazione dei tunnel richiesti dal presente documento. La VM è operativa e configurata in conformità con i requisiti architetturali descritti nelle sezioni precedenti (OS, rete, routing, NAT, sysctl). Il fornitore non dovrà eseguire alcuna installazione di base: il punto di partenza è una VM già operativa, sulla quale installerà esclusivamente il proprio software tunnel.

18.1 Caratteristiche della VM

Parametro	Valore
Sistema operativo	Debian 12 (Bookworm) x86_64
vCPU	2
RAM	4 GB
Disco	20 GB
Init system	systemd
Network manager	systemd-networkd
Firewall	nftables
Kernel	≥ 6.1 , CONFIG_TUN=m, CONFIG_NF_TABLES=y
Utente	satcom (membro del gruppo sudo)

18.2 Configurazione di rete già presente

La VM è configurata con systemd-networkd. Tutti i file si trovano in /etc/systemd/network/.

Interfacce MGMT (statiche)

File	Interfaccia	IP	Scopo
01-mgmt1.network	enp6s18	10.10.11.100/24	Gestione primaria (SSH)
02-mgmt2.network	enp6s19	10.10.10.100/24	Gestione secondaria

Interfacce WAN (DHCP)

File	Interfaccia	WAN
10-wan1.network	enp7s3	WAN1
11-wan2.network	enp7s4	WAN2
12-wan3.network	enp7s5	WAN3
13-wan4.network	enp7s6	WAN4
14-wan5.network	enp7s7	WAN5
15-wan6.network	enp7s8	WAN6

Interfacce LAN (statiche point-to-point)

File	Interfaccia	IP VM
20-lan1.network	enp6s20	172.16.1.1/30
21-lan2.network	enp6s21	172.16.2.1/30
22-lan3.network	enp6s22	172.16.3.1/30
23-lan4.network	enp6s23	172.16.4.1/30
24-lan5.network	enp7s1	172.16.5.1/30
25-lan6.network	enp7s2	172.16.6.1/30

VLAN e bonding Le sotto-interfacce VLAN (§ 5) e il dispositivo di bonding bd1 (VLAN 17 su enp6s20) sono definiti nei corrispondenti file .netdev e .network già presenti in /etc/systemd/network/.

Policy routing Le tabelle di routing personalizzate sono già definite in /etc/iproute2/route_tables:

```
100 wan1    101 wan2    102 wan3    103 wan4    104 wan5    105 wan6
120 mt_cr4  121 mt_br4  122 mt_df4
123 mt_cr5  124 mt_br5  125 mt_df5
126 mt_cr6  127 mt_br6  128 mt_df6
200 bd1
```

NAT e IP forwarding Il file /etc/nftables.conf contiene le regole masquerade per tutte le interfacce WAN e TUN (comprese mpq1-mpq6, mp1, tun-cr/br/df4-6) secondo quanto descritto in § 8.

Il forwarding IPv4 è abilitato persistentemente via /etc/sysctl.d/99-sysctl.conf:

```
net.ipv4.ip_forward = 1
```

I buffer socket sono dimensionati per link satellitari ad alta latenza in /etc/sysctl.d/99-socket-buffers.conf:

```
net.core.rmem_max = 7340032
net.core.wmem_max = 7340032
```

18.3 Accesso alla VM — Fase iniziale (rete locale)

Nella fase iniziale di configurazione, la VM è raggiungibile via SSH sulla rete di gestione primaria:

Host: 10.10.11.100

Porta: 22

Utente: satcom

L'accesso è garantito via rete locale o tramite VPN di laboratorio. La password sarà comunicata separatamente da Telespazio.

18.4 Accesso remoto tramite ZeroTier

Per l'accesso remoto alla VM al di fuori della rete di laboratorio, Telespazio utilizza **ZeroTier** — una rete overlay L2/L3 cifrata che emula una LAN virtuale tra tutti i nodi.

Il fornitore dovrà installare il client ZeroTier sulla VM e fare il join alla rete TRINA il cui **Network ID** sarà comunicato da Telespazio.

```
# 1. Installa ZeroTier (script ufficiale, verifica sempre l'hash prima di eseguire)
curl -s https://install.zerotier.com | sudo bash

# In alternativa, via repository apt (metodo raccomandato per ambienti di produzione)
curl -fsSL https://raw.githubusercontent.com/zerotier/ZeroTierOne/main/doc/contact9
| sudo gpg --dearmor -o /usr/share/keyrings/zerotier.gpg
echo "deb [signed-by=/usr/share/keyrings/zerotier.gpg] https://download.zerotier.com
| sudo tee /etc/apt/sources.list.d/zerotier.list
sudo apt update && sudo apt install -y zerotier-one

# 2. Abilita e avvia il servizio
sudo systemctl enable --now zerotier-one

# 3. Verifica che il nodo sia online e annota il Node ID
sudo zerotier-cli info
# Output atteso: 200 info <NODE_ID> <versione> ONLINE
```

Installazione ZeroTier su Debian/Ubuntu

```
# Sostituire <ZEROTIER_NETWORK_ID> con il valore comunicato da Telespazio
sudo zerotier-cli join <ZEROTIER_NETWORK_ID>

# Verifica stato join
sudo zerotier-cli listnetworks
# Output atteso: <NETWORK_ID> trina <IP_ASSEGNATO>/16 OK PRIVATE ether
```

Join alla rete TRINA

Nota: la rete TRINA è di tipo **PRIVATE**. Il join non è automatico: un amministratore Telespazio deve autorizzare il Node ID del fornitore dal pannello di controllo ZeroTier Central. Comunicare il proprio Node ID (output di `zerotier-cli info`) a Telespazio per procedere con l'autorizzazione.

Verifica connettività Dopo l'autorizzazione, l'interfaccia `ztXXXXXXXXXX` (nome generato automaticamente) apparirà tra le interfacce di rete con un IP nella subnet `10.202.0.0/16`:

```
ip addr show | grep zt
# oppure
sudo zerotier-cli listnetworks

# Test ping verso un nodo della rete TRINA (indirizzo comunicato da Telespazio)
ping 10.202.x.x
```

Accesso SSH via ZeroTier Una volta che la VM è membro autorizzato della rete TRINA, sarà raggiungibile via SSH all'indirizzo ZeroTier assegnatole (nella subnet `10.202.0.0/16`):

```
ssh satcom@10.202.X.X
```

L'indirizzo IP ZeroTier assegnato alla VM verrà comunicato da Telespazio dopo l'autorizzazione, oppure è visibile con `sudo zerotier-cli listnetworks`.

Persistenza al riavvio Il servizio `zerotier-one` è abilitato tramite `systemd` e si avvia automaticamente al boot. Il join alla rete è persistente: non è necessario rieseguire `zerotier-cli join` dopo il riavvio.

18.5 Riepilogo credenziali e riferimenti

Informazione	Valore / Fonte
IP MGMT VM	10.10.11.100 (rete locale laboratorio)
Utente SSH	satcom
Password SSH	Comunicata separatamente da Telespazio
ZeroTier Network ID	Comunicato separatamente da Telespazio
Subnet ZeroTier	10.202.0.0/16
Autorizzazione ZeroTier	Richiesta — inviare il proprio Node ID a Telespazio

Fine del documento.